

עמוד שער

סופר מריו — משחק פלטפורמה עם בינה מלאכותית ## פרויקט גמר במדעי המחשב
10 יחידות לימוד --- **שם התלמיד: ** עמית אלגבסי **כיתה: ** מוסד חינוכי: ** בית
הספר התיכון **שם המנחה: ** שנת לימודים: ** תשפ"ו — 2026 --- *הוגש כחלק ממילוי
דרישות הבגרות במדעי המחשב — 10 יחידות לימוד*

תיאור קצר

פרויקט זה הוא משחק פלטפורמה בסגנון **סופר מריו** הבנוי ב-C# ו-WinForms. המשחק
כולל:

- **שלושה שלבים ידניים** + שני שלבים נוצרים פרוצדורלית
- **שישה סוגי אויבים** עם בינה מלאכותית מגוונת
- **מנגנון כוח-על** (פטרייה), מטבעות, ובלוקי שאלה
- **מנוע פיזיקה** מדויק עם התנגשויות AABB
- **סוכן בינה מלאכותית** — לואיג'י לומד לשחק בעצמו דרך **Neuroevolution** (רשת
נוירונים + אלגוריתם גנטי)

מבנה התיק

פרק	נושא	עמודים (משוער)
1	מבוא ורקע	3
2	הגדרת הבעיה	2

פרק	נושא	עמודים (משוער)
3	ניתוח	5
4	תיכון	8
5	מימוש	10
6	בדיקות	4
7	הוראות הפעלה	3
8	סיכום	2
9	ביבליוגרפיה	1

פרויקט זה הוא **משחק פלטפורמה** בסגנון הקלאסי "סופר מריו", הממומש בשפת #C תחת מסגרת ה-WinForms של .NET. המשחק כולל מנגנוני פיזיקה מדויקים, שישה סוגי אויבים, שלושה שלבים עיצוביים ידניים ושניים נוצרים אוטומטית, פריטי כוח-על ומערכת ניקוד. הייחוד הבולט בפרויקט הוא **רכיב הבינה המלאכותית**: סוכן בשם לואיג'י לומד לשחק במשחק בעצמו ללא תכנות ידני של הכללים. הלמידה מתבצעת באמצעות **Neuroevolution** — שילוב של **רשת נוירונים ואלגוריתם גנטי**. אוכלוסייה של 60 סוכנים רצים בו-זמנית, כאשר הסוכנים החזקים ביותר מעבירים את ה"גנים" שלהם לדור הבא. לאחר מספר דורות הסוכן לומד לקפץ על מכשולים, לחצות פלטפורמות ולהגיע למרחקים גדולים יותר ויותר.

תוצאות עיקריות:

- משחק פלטפורמה פונקציונלי ומהנה עם תחושת תנועה אותנטית
- מנוע פיזיקה עצמאי עם תנגשויות, כוח משיכה וקפיצות משתנות
- סוכן לואיג'י המסוגל ללמוד לשחק מאפס ללא הנחיות מפורשות
- ממשק אימון חי המציג את הדור, הניקוד, וויזואליזציה של מבנה הרשת

שפות וכלים: C#, .NET Framework 4.7.2, WinForms, GDI+, Python (לייזור sprites).

מילות מפתח: פיתוח משחקים, WinForms, C#, פיזיקת פלטפורמה, AAB, רשת נוירונים, אלגוריתם גנטי, Neuroevolution, בינה מלאכותית

1.1 רקע

משחקי פלטפורמה הם אחד הז'אנרים הוותיקים בתולדות תעשיית המשחקים. **Super Mario Bros** של Nintendo (1985) הגדיר את התבנית: שחקן הקופץ על פלטפורמות, נמנע מאויבים, ואוסף פריטים לאורך שלבים. מאחורי הפשטות הכאילו-פעוטה מסתתר מנגנון פיזיקה מדויק — כוח משיכה, תאוצה, עצירה הדרגתית, קפיצה עם גובה משתנה לפי אורך הלחיצה.

בפרויקט זה בחרתי לבנות מחדש את חוויית המשחק הקלאסי הזו בעצמי, תוך שאני מאמת מתמטית כל קבוע פיזיקלי ובונה ארכיטקטורה נקייה. בנוסף, הרחבתי את הפרויקט בכיוון שלא קיים במשחק המקורי: **בינה מלאכותית** שלומדת לשחק מאפס.

1.2 מוטיבציה

שלוש שאלות הניעו את הפרויקט:

1. **כיצד מיישמים פיזיקה אמינה של פלטפורמה?** — לא להשתמש בספרייה מוכנה, אלא לבנות בעצמי מנוע המסוגל לנהל קפיצות, תאוצה, התנגשויות ומוות.
2. **כיצד מפתחים משחק ברמה מקצועית ב-C# ללא מנוע משחקים?** — להשתמש ב-WinForms וב-GDI+ בלבד, תוך ניהול קצב פריימים, עיבוד ספרייט-שיטים ואנימציה.
3. **האם ניתן ללמד מחשב לשחק ללא תכנות ידני של הכללים?** — לממש Neuroevolution: הגדיר לסוכן קלטים חישתיים ומטרה, ותן לאלגוריתם הגנטי למצוא את המשקולות הנכונות.

1.3 מטרות הפרויקט

מטרה	תיאור
פונקציונלית	בנות משחק פלטפורמה ניתן למשחק עם שלבים, אויבים ופריטים

מטרה	תיאור
הנדסית	מנוע פיזיקה עצמאי, ארכיטקטורה מודולרית, ביצועים טובים
בינה מלאכותית	סוכן לומד שמשפר ביצועיו דור אחר דור ללא התערבות ידנית
ויזואלית	ספרייט-שיטים אותנטיים, אנימציות, תפריט מונפש

1.4 נושאים שנלמדו במסגרת הפרויקט

מתכנות

- תכנות מונחה-עצמים (OOP) מתקדם: מחלקות חלקיות (`partial class`), ממשקים, ירושה
- ניהול ריבוי אירועים (event-driven programming) ב-WinForms
- ניהול גרפיקה עם GDI+ ו- `Graphics`
- טיפול בזיכרון: `Dispose` , `using` , הימנעות מדליפות
- ביצועים: `SuspendLayout` / `ResumeLayout` , קאש, צמצום `redraws`

אלגוריתמים ומתמטיקה

- פתרון התנגשויות (Axis-Aligned Bounding Box) AABB
- פיזיקה: אינטגרציה מספרית, כוח משיכה, תאוצה
- רשתות נוירונים: Forward Pass , `tanh` activation
- אלגוריתם גנטי: elitism, crossover, mutation

כלים ותהליכי פיתוח

- ניהול גרסאות עם Git (ענפים, merge, PR)
- Python לייצור נכסים (pixel art, sprite sheets)

1.5 מגבלות הפרויקט

- המשחק רץ על **Windows בלבד** (WinForms)
- אין שמירת התקדמות בין הפעלות

- הבינה המלאכותית לומדת בסביבת אימון פשוטה (ללא אויבים, ללא מטבעות)
- לא מיושמת מוסיקת רקע (רק גרפיקה)

 **צילום מסך:** תפריט ראשי של המשחק: רקע מונפש, לוגו "4", "Super Mario" כפתורים

1.6 מבנה המסמך

פרק 2 מגדיר את הבעיה. פרק 3 מנתח את הדרישות והמודל. פרק 4 מציג את תיכון המערכת. פרק 5 מפרט את המימוש. פרק 6 מסכם את הבדיקות. פרק 7 הוא מדריך למשתמש. פרק 8 סיכום ומסקנות.

הגדרת הבעיה

2.1 מהי הבעיה?

פרויקט זה עוסק בשתי בעיות מרכזיות שלכל אחת מהן ייחוד תכנותי:

בעיה א' — בניית משחק פלטפורמה אותנטי

בניית משחק פלטפורמה בדרגת איכות ראויה ללא שימוש במנוע משחקים מוכן (Unity, Godot וכד'), תוך שמירה על:

- **תנועה אמינה:** תחושה שמרבית גיימרים מזהים כ"נכונה" — תאוצה הדרגתית, עצירה חלקה, קפיצה עם גובה משתנה
- **אויבים חכמים:** כל אויב מתנהג שונה — הולך, קופץ, סייר על פלטפורמה, טס
- **שלבים מעוצבים:** שלושה שלבים שונים בסגנון + שניים פרוצדורליים
- **ביצועים:** 60 פריימים לשנייה ללא תקיעות, גם כאשר מאות אובייקטים על המסך

בעיה ב' — בינה מלאכותית ללא פקודות ידניות

בהינתן סוכן בעולם הפלטפורמה, **כיצד ניתן ללמד אותו לשחק** — להגיע כמה שיותר ימינה — מבלי לכתוב לו במפורש "קפץ כשיש פי", "הזז ימינה"?
האתגר: הסוכן קולט רק 4 ערכים מספריים (ראו פרק 5), ועליו להחליט בכל 16ms — לאן לנוע, האם לקפץ. מרחב ההחלטות הוא אינסופי, ופונקציית הציון (fitness) פשוטה בכוונה: כמה ימינה הגעת?

2.2 למה זה מורכב?

מורכבות הפיזיקה

פיזיקה פשוטה (מיקום + מהירות) מייצרת תנועה "מרחפת" ולא טבעית. המשחק המקורי של Nintendo עובד עם **תאוצות ועצירות שונות באוויר ועל הקרקע**. כדי לשכפל זאת:

כיוון $\times \text{MoveSpeed} = \text{הכוח הנגד}$
באוויר (0.42) AirAccel , בקרקע (0.75) GroundAccel = התאוצה אל הכוח
באוויר (0.16) AirDecel , בקרקע (0.55) GroundDecel = העצירה

כלומר, ישנם 6 קבועים פיזיקליים שיש לכוון בקפידה כדי שהמשחק ירגיש נכון.

מורכבות ההתנגשויות

כל פריים, כל אובייקט נע (שחקן, אויבים, פטריות, מטבעות) עלול להיכנס לתוך פלטפורמה. פתרון "תקוע בתוך קיר" דורש:

1. זיהוי כל 4 כיווני חדירה
2. בחירת הכיוון הקצר ביותר להוצאה
3. טיפול בפינות (corner cases) כמו נחיתה תוך קפיצה בו-זמנית

מורכבות הבינה המלאכותית

מרחב הפרמטרים של רשת נוירונים עם 4 כניסות, 2 שכבות נסתרות (6 ו-4 נוירונים) ו-2 יציאות = כ-60 משקולות. לא ניתן לחפש בצורה ממצה. הפתרון: **אלגוריתם גנטי** שמאמץ את הפתרונות הטובים ומשמיד את הרעים.

2.3 מי המשתמשים?

משתמש	מה הוא רוצה
שחקן	להנות ממשחק פלטפורמה חלק עם אתגר הולך וגדל
חוקר / מסקרן	לצפות בסוכן הלומד ולשנות פרמטרים (גודל אוכלוסייה, אחוז מוטציה)
מפתח	לקרוא קוד ברור, מודולרי, עם ארכיטקטורה מוסברת

2.4 הנחות המוצא

1. ה-PC מריץ Windows עם NET Framework 4.7.2
2. המשחק אינו מרובה משתתפים — שחקן יחיד
3. הבינה המלאכותית לומדת מאפס בכל הפעלה (אין שמירת מצב ב-commit הנוכחי)
4. האנימציה מבוססת sprite-sheets — קבצי PNG חיצוניים בתיקייה `/assets`

3.1 דרישות המערכת

דרישות פונקציונליות

#	דרישה	עדיפות
F1	השחקן נע שמאלה/ימינה בעזרת מקלדת	חובה
F2	השחקן קופץ בגובה משתנה לפי אורך הלחיצה	חובה
F3	ישנם לפחות 3 שלבים שונים	חובה
F4	ישנם לפחות 4 סוגי אויבים	חובה
F5	השחקן יכול לדרוך על אויבים ולחסל אותם	חובה
F6	קיימים מטבעות, פטריות ובלוקי שאלה	חובה
F7	קיים תפריט ראשי מונפש	חובה
F8	קיימת מערכת ניקוד וחיים	חובה
F9	קיים מצב אימון בינה מלאכותית	רצוי
F10	ויזואליזציה חיה של רשת הנזירונים	רצוי
F11	שמירת רשת הנזירונים לקובץ	אופציונלי

דרישות לא-פונקציונליות

#	דרישה	ערך
NF1	קצב פריימים	$\text{fps } 60 \leq$
NF2	עיכוב תגובה לקלט	$\text{ms } 16 >$
NF3	צריכת זיכרון	$\text{MB } 150 >$

#	דרישה	ערך
NF4	זמן טעינה	> 3 שניות
NF5	תמיכת מערכת הפעלה	+Windows 7

3.2 מודל נתונים

ישויות מרכזיות

```

Player (שחקן)
├─ Health: int = 3
├─ Score: int
├─ IsGrounded: bool
├─ IsSuper: bool (גדול/קטן)
├─ Position: Point
├─ HorizontalVelocity: float
├─ VerticalVelocity: float

Enemy (אויב - בסיס משותף)
├─ Position: Point
├─ IsAlive: bool
├─ Direction: int (-1 / +1)
├─ [טיפוס-ספציפי]

GameObjectS (פלטפורמה/צ'ינוור)
├─ Visual: PictureBox
├─ Bounds: Rectangle (world-space)
├─ IsGround: bool

GameData – GameObjects (פריטי משחק)
├─ Mushroom
│   ├── Position, IsActive
│   └── VelocityX, VelocityY
├─ Coin
│   ├── Position, IsActive
│   └── AnimFrame
├─ QuestionBlock
│   ├── Position, IsActivated
│   └── PowerUpType (Coin / Mushroom)

```

מודל הבינה המלאכותית

```
NetParams (הגדרות גלובליות)
├─ PopulationSize: int = 60
├─ MutationRate: float = 0.05
├─ SurviveRate: float = 0.30
└─ NetworkShape: int[] = {4, 6, 4, 2}

NeuralNetwork (מוח של סוכן)
├─ Layers: Layer[]
└─ Forward(inputs: float[]) → float[]

MarioAgent (סוכן לואיג'י)
├─ Brain: NeuralNetwork
├─ Position: Point
├─ IsAlive: bool
├─ Fitness: float (= שהושג X מקסימום)
└─ ComputeInputs(world) → float[4]

Population (אוכלוסייה)
├─ Agents: MarioAgent[]
├─ Generation: int
└─ CreateNewGeneration() – elitism + crossover + mutation
```

3.3 מודל הפיזיקה

קבועים פיזיקליים

קבוע	ערך	יחידות	תפקיד
MoveSpeed	4.4	פיקסל/תק	מהירות יעד בריצה
MaxMoveSpeed	5.6	פיקסל/תק	תקרת מהירות אופקית
GroundAccel	0.75	פיקסל/תק ²	תאוצה על הקרקע
AirAccel	0.42	פיקסל/תק ²	תאוצה באוויר
GroundDecel	0.55	פיקסל/תק ²	עצירה על הקרקע
AirDecel	0.16	פיקסל/תק ²	עצירה באוויר
Gravity	0.58	פיקסל/תק ²	כוח משיכה

קבוע	ערך	יחידות	תפקיד
JumpPower	13.8-	פיקסל/תק	עוצמת קפיצה (שלילי = למעלה)
MaxFallSpeed	15.5	פיקסל/תק	מהירות נפילה מקסימלית
JumpReleaseGravity	2.4×	—	מכפיל כוח משיכה בשחרור מקש

הערה: tick = 16ms, כלומר ~60 ticks לשנייה.

אלגוריתם אינטגרציה (בכל tick)

1. $horizontalVelocity \leftarrow Approach(currentVX, target, accel)$ חשב
2. $[-MaxMoveSpeed, +MaxMoveSpeed]$ -קצץ ל
3. $preciseX += horizontalVelocity$ הזז
4. גבול עולם: $clamp\ X \in [0, 2950]$
5. AND IsGrounded: $VerticalVelocity = -13.8$ אם קפיצה נלחצה
6. $VerticalVelocity += gravity (\times 2.4)$ (אם קפיצה שוחררה $\times 2.4$)
7. קצץ: $VerticalVelocity \leq 15.5$
8. הזז: $preciseY += VerticalVelocity$
9. עגל: $Position = (round(preciseX), round(preciseY))$

אלגוריתם התנגשות (AABB — ResolveSmallestOverlap)

- (plat) חופף פלטפורמה (body) נתון: גוף נע
1. חשב 4 חדירות:

$ot = body.Bottom - plat.Top$	(חדירה מלמעלה)
$ob = plat.Bottom - body.Top$	(חדירה מלמטה)
$ol = body.Right - plat.Left$	(חדירה משמאל)
$or = plat.Right - body.Left$	(חדירה מימין)
 2. בחר מינימום מבין הארבע
 3. $ot == ot \text{ AND } ot < 30px$:
 - נחיתה: $SnapAbove(plat.Top), VY = 0, IsGrounded = true$
 - אם $ob == ob$:
 - פגיעה בתקרה: $SnapBelow(plat.Bottom), VY = 0$
 - אם $ol == ol$:
 - חסימה מצד שמאל

or == אם מינימום
→ חסימה מצד ימין

ספירה: סף 30px (לא 25px) מבטיח שגוף הנופל ב-15.5px/tick לא יחמיק דרך הרצפה.

3.4 מודל הבינה המלאכותית

ארכיטקטורת רשת הנוירונים



כל נוירון מחשב: $output = \tanh(\sum(input_i \times weight_i) + bias)$

אלגוריתם גנטי

דור 1:

רשתות עם משקולות אקראיות 60

בכל דור:

- כל סוכן חי רץ עד שנופל / נתקע / מגיע לסוף
- השיג $fitness(X) = \max(X)$ (סוכן)
- $fitness$ מייין סוכנים לפי
- (elitism) שמור 30% עליונים
- מלא את שאר האוכלוסייה:
 - בחר אב ואם מהשורדים
 - לכל משקולה בחר מאיזה הורה: Crossover
 - בהסתברות 5% – שנה משקולה אקראית: Mutation
- חזור

קריטריון עצירה: לא אוטומטי – המשתמש עוצר ידנית

 **צילום מסך:** 4 סוגי אויבים על המסך בו-זמנית: גומבה, קופה, JumpingEnemy, FlyingEnemy

3.5 סוגי האויבים

אויב	תנועה	תגובה לדריכה	נזק
Goomba	הולך, הופך בקיר	ריסוק → נעלם	1-HP
Koopa	הולך לאט	צדף (נעלם)	1-HP
FastEnemy	מהירות 2×	ריסוק → נעלם	1-HP
JumpingEnemy	הולך + קופץ כל 1.5~ שני'	ריסוק → נעלם	1-HP
PlatformPatrolEnemy	מסייר, מתהפך בקצה פלטפורמה	ריסוק → נעלם	1-HP
FlyingEnemy	טיסה בסינוס	דריכה 1: מאבד כנפיים; 2: נעלם	1-HP

4.1 ארכיטקטורה כללית

המערכת מחולקת ל-6 שכבות עיקריות:

שכבה 1 – כניסה לתוכנית
Program.cs → Application.Run(MainMenuForm)

שכבה 2 (Forms) – ממשק משתמש
MainMenuForm ← תפריט ראשי מונפש
mainWin ← (7 partial classes) לולאת המשחק הראשית
TrainingForm ← ממשק אימון הבינה המלאכותית

שכבה 3 – ליבת המשחק
Core/Player ← super פיזיקה, בריאות, מצב
Core/GameManager ← מצב משחק (Running/Paused/Dead/Win)
Core/GameData ← Mushroom, Coin, QuestionBlock הגדרות
Core/TextureLoader ← sprite-sheets טעינת

שכבה 4 – ישויות
Enemies/* ← סוגי אויבים 6
World/GameObjects ← פלטפורמות וצינורות

שכבה 5 – נכסים
assets/textures/sprite_sheets/*.png

שכבה 6 – בינה מלאכותית
ML/NetParams
ML/Neuron, Layer, NeuralNetwork
ML/MarioAgent
ML/Population
ML/NeuralNetworkControl

4.2 פירוק mainWin ל-7 Partial Classes

mainWin היא ה-Form הראשית. כדי להימנע מקובץ ענק, היא מחולקת לקבצים:

קובץ	אחריות
mainWin.cs	שדות, לולאת משחק, קלט מקלדת
mainWin.LevelData.cs	מערכי שלבים, הגדרות בלוקי שאלה, templates פרוצדורליים
mainWin.LevelBuilder.cs	בניית שלב, ניקוי שלב, יצירת פלטפורמות
mainWin.Physics.cs	פתרון התנגשויות, מצלמה, מוות
mainWin.EnemyUpdates.cs	spawn והעדכון של כל 6 סוגי האויבים
mainWin.Collectibles.cs	עדכון מטבעות ופטריות
mainWin.HUD.cs	שרטוט sprite השחקן, overlay ניקוד/חיים

כל 7 הקבצים מתקמפלים לכיתה אחת — `partial class` של `C#`.

4.3 תרשים זרימה — לולאת המשחק (Game Loop)

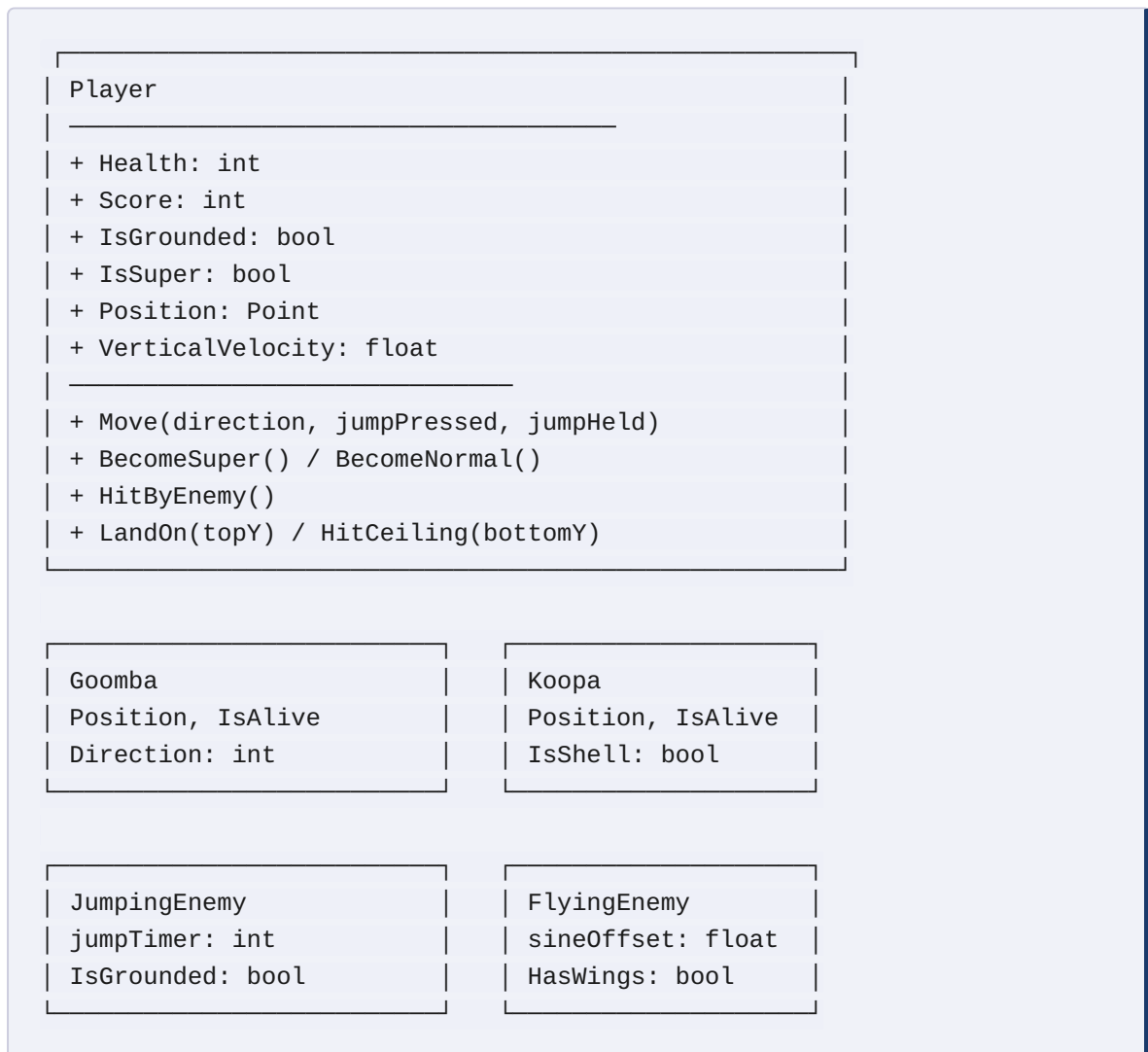
```

gameTimer 16 ms:
|
▼
PhysicsStep()
├─ player.Move(direction, jumpEdge, jumpHeld)
├─ CheckPlatformCollisions() [שחקן מול כל פלטפורמה]
├─ UpdateGoombas()
├─ UpdateKoopas()
├─ UpdateFastEnemies()
├─ UpdateJumpingEnemies()
├─ UpdatePatrolEnemies()
├─ UpdateFlyingEnemies()
├─ UpdateCoins()
└─ UpdateMushrooms()
|
▼
UpdateCamera() → bool (האם המצלמה זזה)
|
├─ Invalidate() [ציור מחדש] אם כן
|
▼
UpdateHud() [עדכון תוויות ניקוד/חיים]

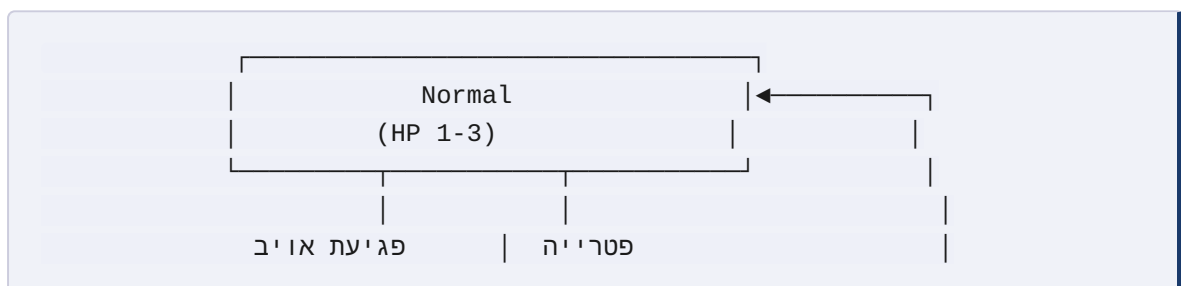
```

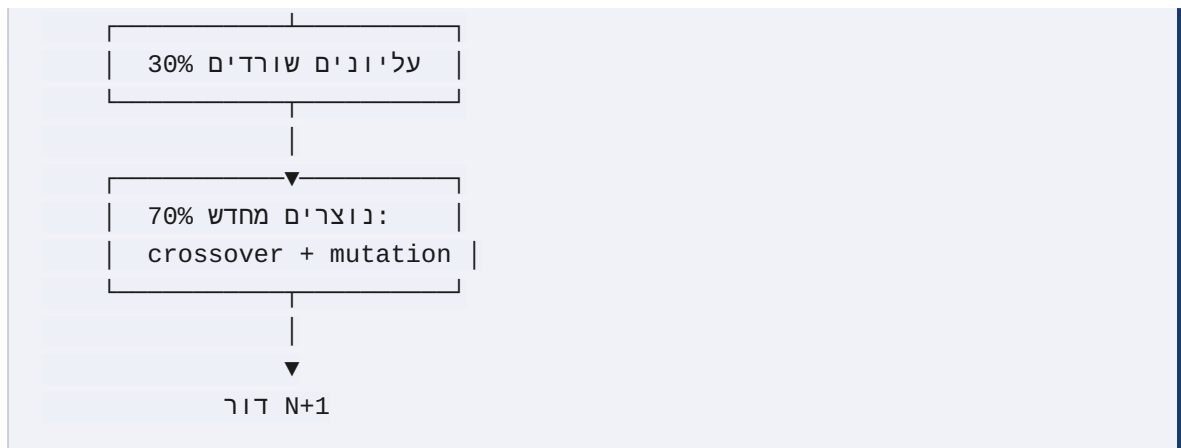

אופטימיזציית ביצועים מפתח: `()SuspendLayout` נקרא פעם אחת בתחילת כל `PhysicsStep` - `()ResumeLayout` בסוף — במקום 7 פעמים בתוך כל תת-פונקציה. חסכון: ~7× קריאות layout לכל `tick`.

4.4 תרשים מחלקות — ליבת המשחק



4.5 תרשים מצבים — מחזור חיי השחקן





4.7 תרשים רצף — tick אחד של אימון

```

_simTimer (16ms)
|
▼ לכל סוכן חי:
├─ ComputeInputs(world) → [dist_gap, dist_enemy, height_diff, grounded]
├─ Brain.Forward(inputs) → [dir_tanh, jump_bool]
├─ Agent.Step(dir, jump) [פיזיקה זהה לשחקן]
├─ ApplyPlatformCollisions(agent)
|
▼ לאחר עדכון כולם:
├─ לפי הסוכן המוביל _cameraX עדכן
├─ CreateNewGeneration() אם כולם מתו
├─ Dashboard (חיים, דור, best) עדכן
├─ _canvas.Invalidate()
  
```

 **צילום מסך:** ממשק ה-Training Form: חלונית האימון עם ה-canvas משמאל (60 לואיג'ים רצים) ולוח הבקרה מימין (דורות, ויזואליזציה של רשת)

4.8 ניהול Z-Stack (סדר ציור)

ב-WinForms, סדר ציור הקונטרולים קובע מה יופיע מעל מה:

(עליון - נראה ראשון)
 HUD labels (חיים, מטבעות, ניקוד)
 Player (picboxplayer)
 Enemies (6 סוגים)
 Coins
 Question Blocks
 Platforms
 Bricks / ground strip
 (תחתון - רקע)
 Form background

באג היסטורי שתוקן: קריאות `SendToBack()` בפונקציות spawn של אויבים גרמו לאויבים להיות מוסתרים מאחורי הרצפה. הפתרון: הסרת ה-`SendToBack` -
`BringToFront()` לשחקן בלבד.

4.9 אסטרטגיית ה-Sprite Sheets

במקום לטעון PNG נפרד לכל פריים אנימציה, כל האנימציות נמצאות ב-5 גיליונות:

גודל	תוכן	גיליון
64×256	idle, walk×2, jump	player_sheet.png
96×256	Goomba×2, Koopa×2	enemies_sheet.png
64×320	coin×4, mushroom	items_sheet.png
64×320	Q-block×2, empty, brick	blocks_sheet.png
128×256	שמים, עננים, גבעות	world_bg.png

צילום מסך: sprite sheet נפתח בתוכנת תמונות: רואים את 4 הפריימים של השחקן זה לצד

זה

לוגיקת אנימציה: בכל tick ה- `globalTick` עולה ב-1. פריים ההליכה מחושב לפי

`2 % 10 / globalTick`.

5.1 סביבת הפיתוח

כלי	גרסה	שימוש
Visual Studio	2022	סביבת פיתוח ראשית
#C	NET Framework 4.7.2.	שפת תכנות
WinForms	—	ממשק גרפי
+GDI	—	ציור 2D
Python	x.3	ייצור pixel art ו-sprite sheets
Git	—	ניהול גרסאות

5.2 מבנה תיקיות הפרויקט

```

supermario/
├── supermario.sln
├── assets/
│   ├── textures/sprite_sheets/
│   │   ├── player_sheet.png
│   │   ├── enemies_sheet.png
│   │   ├── items_sheet.png
│   │   ├── blocks_sheet.png
│   │   └── world_bg.png
│   ├── generate_pixelart.py
│   └── generate_spritesheets.py
├── supermario/
│   ├── Core/
│   │   ├── Player.cs
│   │   ├── GameManager.cs
│   │   ├── GameData.cs
│   │   └── TextureLoader.cs
│   ├── Enemies/
│   │   └── Goomba.cs
└── C# פרויקט ←
    
```

```

|   |   |— Koopa.cs
|   |   |— FastEnemy.cs
|   |   |— JumpingEnemy.cs
|   |   |— PlatformPatrolEnemy.cs
|   |   |— FlyingEnemy.cs
|   |— World/
|   |   |— GameObjectS.cs
|   |— UI/
|   |   |— MainMenuForm.cs
|   |   |— mainWin.cs
|   |   |— mainWin.LevelData.cs
|   |   |— mainWin.LevelBuilder.cs
|   |   |— mainWin.Physics.cs
|   |   |— mainWin.EnemyUpdates.cs
|   |   |— mainWin.Collectibles.cs
|   |   |— mainWin.HUD.cs
|   |   |— TrainingForm.cs      ← [ענף לוויג'י]
|   |— ML/                      ← [ענף לוויג'י]
|   |   |— NetParams.cs
|   |   |— Neuron.cs
|   |   |— Layer.cs
|   |   |— NeuralNetwork.cs
|   |   |— NeuralNetworkControl.cs
|   |   |— MarioAgent.cs
|   |   |— Population.cs

```

5.3 מנגנון הפיזיקה — קטעי קוד מרכזיים

פונקציית Approach (תאוצה חלקה)

```

static float Approach(float current, float target, float rate)
{
    if (current < target) return Math.Min(current + rate, target);
    if (current > target) return Math.Max(current - rate, target);
    return current;
}

```

פונקציה זו מייצרת תנועה חלקה: הגוף מתקרב למהירות היעד בקצב קבוע, ללא חריגה.

תאוצה אופקית (תוך Player.Move)

```
float target = direction * MoveSpeed;
float accel  = IsGrounded ? GroundAcceleration : AirAcceleration;
float decel  = IsGrounded ? GroundDeceleration : AirDeceleration;

horizontalVelocity = (direction != 0)
    ? Approach(horizontalVelocity, target, accel)
    : Approach(horizontalVelocity, 0f, decel);

horizontalVelocity = Math.Max(-MaxMoveSpeed,
    Math.Min( MaxMoveSpeed, horizontalVelocity));
preciseX += horizontalVelocity;
```

קפיצה משתנה

```
if (jumpPressed && IsGrounded)
{
    VerticalVelocity = JumpPower;    // -13.8
    IsGrounded = false;
}

if (!IsGrounded)
{
    float g = Gravity;
    if (!isJumpHeld && VerticalVelocity < 0)
        g *= JumpReleaseGravityMultiplier;    // 2.4x

    VerticalVelocity = Math.Min(VerticalVelocity + g, MaxFallSpeed);
    preciseY += VerticalVelocity;
}
```

 **צילום מסך:** השחקן באוויר תוך קפיצה: רואים את הספרייט של הקפיצה (פריים 3)

פתרון התנגשות (ResolveSmallestOverlap)

```
float ot = body.Bottom - plat.Top;
float ob = plat.Bottom - body.Top;
```



```
float ol = body.Right - plat.Left;
float orr = plat.Right - body.Left;

float min = Math.Min(Math.Min(ot, ob), Math.Min(ol, orr));

if (min == ot && ot < 30) player.LandOn(plat.Top - bodyHeight);
else if (min == ob) player.HitCeiling(plat.Bottom);
else if (min == ol) player.BlockHorizontal(plat.Left - bodyWidth);
else if (min == orr) player.BlockHorizontal(plat.Right);
```

נקודה חשובה: הסף $ot < 30$ (לא 25) מבטיח שנפילה במהירות מקסימלית של 15.5px/tick לא תעבור דרך הרצפה ב"מנהרה".

5.4 מנגנון האויבים

הורשה וגמישות

כל אויב הוא **struct** או **class** עם שדות ספציפיים. אין מחלקת בסיס (inheritance) — הבחירה הייתה פשוט על פני אבסטרקציה.

קופץ — **JumpingEnemy**

```
private void UpdateJumpingEnemy(JumpingEnemy e)
{
    e.JumpTimer++;
    if (e.JumpTimer >= 90 && e.IsGrounded) // כל 1.5 שני (90 ticks)
    {
        e.VerticalVelocity = JumpPower;
        e.IsGrounded = false;
        e.JumpTimer = 0;
    }
    // פיזיקה זהה לשחקן...
}
```

📷 **צילום מסך:** JumpingEnemy באוויר מעל פלטפורמה, ניכר שהוא שונה מ-Goomba

סייר פלטפורמה — PlatformPatrolEnemy

```
// בדוק אם הפלטפורמה נגמרת קדימה: בכל tick
bool edgeAhead = !PlatformExistsAt(
    e.Position.X + e.Direction * 20,
    e.Position.Y + 5);

if (edgeAhead)
    e.Direction *= -1; // הפוך כיוון
```

5.5 מנגנון הבינה המלאכותית

Forward Pass של רשת נוירונים

```
// NeuralNetwork.cs
public float[] Forward(float[] inputs)
{
    float[] current = inputs;
    foreach (var layer in Layers)
        current = layer.Forward(current);
    return current;
}

// Layer.cs
public float[] Forward(float[] inputs)
{
    float[] outputs = new float[Neurons.Length];
    for (int i = 0; i < Neurons.Length; i++)
        outputs[i] = Neurons[i].Forward(inputs);
    return outputs;
}

// Neuron.cs
public float Forward(float[] inputs)
{
    float sum = Bias;
```

```

for (int i = 0; i < Weights.Length; i++)
    sum += Weights[i] * inputs[i];
return (float)Math.Tanh(sum);
}

```

חישוב קלטים לסוכן

```

// MarioAgent.ComputeInputs()
float distGap      = DistanceToNextGap(world);    // מרחק לחריץ הבא
float distEnemy    = DistanceToNearestEnemy();    // מרחק לאויב הקרוב
float heightDiff   = NextPlatformHeightDiff();    // הפרש גובה פלטפורמה
float grounded     = IsGrounded ? 1f : 0f;        // האם על הקרקע

return new[] { distGap, distEnemy, heightDiff, grounded };

```

יצירת דור חדש (Neuroevolution)

```

// Population.CreateNewGeneration()
var sorted = Agents.OrderByDescending(a => a.Fitness).ToArray();
int survivors = (int)(sorted.Length * SurviveRate);

var nextGen = new MarioAgent[Agents.Length];

// elitism (שמור את הטוב ביותר ישירות)
nextGen[0] = sorted[0].Clone();

for (int i = 1; i < nextGen.Length; i++)
{
    var parent1 = sorted[_rng.Next(survivors)];
    var parent2 = sorted[_rng.Next(survivors)];
    nextGen[i] = Crossover(parent1, parent2);
    Mutate(nextGen[i]);
}
Agents = nextGen;
Generation++;

```

צילום מסך: ויזואליזציה של רשת הנירונים בלוח הבקרה: 4 עמודות של נירונים, קווים כחולים/אדומים למשקולות

Crossover בין שני הורים

```
MarioAgent Crossover(MarioAgent p1, MarioAgent p2)
{
    var child = p1.Clone();
    var w1 = p1.Brain.GetWeights();
    var w2 = p2.Brain.GetWeights();
    var wc = new float[w1.Length];

    for (int i = 0; i < wc.Length; i++)
        wc[i] = (_rng.NextDouble() < 0.5) ? w1[i] : w2[i];

    child.Brain.SetWeights(wc);
    return child;
}
```

מוטציה

```
void Mutate(MarioAgent agent)
{
    var weights = agent.Brain.GetWeights();
    for (int i = 0; i < weights.Length; i++)
        if (_rng.NextDouble() < MutationRate)
            weights[i] += (float)(_rng.NextGaussian() * 0.3);
    agent.Brain.SetWeights(weights);
}
```

5.6 מבנה השלבים

שלבים ידניים

המשחק מכיל 3 שלבים ידניים המוגדרים כמערכי נתונים ב- `mainWin.LevelData.cs`:

```
static readonly int[,] LEVEL_1 = {
    // [x, y, width, height, type]
    { 0, 513, 3000, 60, TYPE_GROUND },
    { 200, 380, 120, 20, TYPE_PLATFORM },
    // ... פלטפורמות
};
```

שלב	נושא	מאפיינים
שלב 1	Overworld (ירוק, שמים)	שלב קל, מלמד בסיסים
שלב 2	Underground (כהה, מחתרת)	צינורות, מרחב צר
שלב 3	Sky Fortress (שמים גבוה)	פלטפורמות צפות, אויבים מעופפים

שלבים פרוצדורליים (4-5)

שלבים 4 ו-5 נוצרים בזמן ריצה מ-25 "תבניות קטעים" (section templates):

```
// כל תבנית = רצף פלטפורמות עם אויבים/פריטים
static readonly SectionTemplate[] TEMPLATES = {
    new SectionTemplate { Platforms = ..., Enemies = ... },
    // ...
};
```

 **צילום מסך:** שלב 1 של המשחק: השחקן (מריו) עומד על הקרקע, פלטפורמות, בלוק שאלה, גומבה

 **צילום מסך:** שלב 3: שלב שמיים עם פלטפורמות צפות ואויב מעופף (FlyingEnemy)

5.7 ביצועים — אופטימיזציות מפתח

אופטימיזציה	לפני	אחרי	השפעה
SuspendLayout יחיד	7 קריאות/tick	1 קריאה/tick	הפחתה של 7×
UpdateCamera מחזיר bool	Invalidate תמיד	רק כשזזה	מסיר repaints מיותרים

אופטימיזציה	לפני	אחרי	השפעה
רצועת קרקע יחידה	PictureBox 75	1 PictureBox	הסרת צוואר בקבוק עיקרי
globalTick % 168	גלישת int לאחר שעות	עטיפה נקייה	מניעת באג אנימציה
BackColor אטום בטיילים	ציור שקוף (~75/ tick)	אפס	ביטול repaints של הורה

5.8 גנרטור ה-Pixel Art (Python)

שני סקריפטים Python יוצרים את כל נכסי המשחק:

```
# generate_pixelart.py – יוצר תמונות בודדות
def draw_mario_idle(scale=4):
    img = Image.new('RGBA', (16*scale, 16*scale))
    # ... ציור פיקסל-אחר-פיקסל ...
    return img

# generate_spritesheets.py – מרכיב לגיליונות
def build_player_sheet():
    sheet = Image.new('RGBA', (256, 64))
    frames = [idle, walk1, walk2, jump]
    for i, frame in enumerate(frames):
        sheet.paste(frame, (i*64, 0))
    sheet.save('assets/textures/sprite_sheets/player_sheet.png')
```

בדיקות

6.1 אסטרטגיית הבדיקות

הבדיקות בפרויקט זה הן **בדיקות ידניות** (manual testing), מאחר שאין ב-WinForms תשתית בדיקות אוטומטיות מובנית. הבדיקות מחולקות ל:

1. **בדיקות יחידה (Unit)** — בדיקת פונקציות פיזיקה ומתמטיקה בבידוד
2. **בדיקות אינטגרציה** — בדיקת אינטראקציות בין מודולים
3. **בדיקות מערכת** — שחקן שמשחק את השלבים מתחילה לסוף
4. **בדיקות קצה (Edge Cases)** — מצבים קיצוניים שגרמו לבאגים

6.2 תכנית בדיקות — פיזיקה

#	מה נבדק	תנאי	תוצאה צפויה	תוצאה בפועל
P1	קפיצה בסיסית	לחיצת Space	השחקן עולה ויורד	עובד ✓
P2	קפיצה קצרה	לחיצה קצרה מאוד	גובה נמוך מקפיצה מלאה	עובד ✓
P3	קפיצה מלאה	לחיצה ממושכת	גובה מקסימלי	עובד ✓
P4	נחיתה על פלטפורמה	קפיצה מעל פלטפורמה	נחיתה מעל, לא חדירה	עובד ✓
P5	מניעת "מנהרה"	נפילה מגובה רב	לא עובר דרך הרצפה	תוקן בקומיט 1e82bb3 ✓
P6	חסימת קיר	הליכה לתוך קיר	עצירה, לא חדירה	עובד ✓
P7	פגיעה בתקרה	קפיצה לתוך בלוק	$VY=0$, גוף לא נכנס	עובד ✓
P8	כוח משיכה לאויבים	אויב נדחף מפלטפורמה	נופל בצורה חלקה	תוקן בקומיט c8edfbb ✓

#	מה נבדק	תנאי	תוצאה צפויה	תוצאה בפועל
P9	נפילה לבור	השחקן עובר Y=580	מוות מידי	עובד ✓
P10	נזק נפילה	נפילה מגובה 220px<	HP 1-	תוקן בקומיט 3cdb3fe ✓

6.3 תכנית בדיקות — אויבים

#	מה נבדק	תנאי	תוצאה צפויה	תוצאה בפועל
E1	Goomba — הפיכה בקיר	הגומבה מגיע לקיר	הופך כיוון	עובד ✓
E2	Goomba — דריכה	קפיצה על הגומבה	נריסוק + 100+ ניקוד	עובד ✓
E3	Goomba — פגיעה בשחקן	מגע מהצד	HP 1-	עובד ✓
E4	JumpingEnemy — קצב	ספירת זמן	קופץ כל ~1.5 שני	תוקן בקומיט d208cb7 ✓
E5	PlatformPatrol — קצה	מגיע לקצה פלטפורמה	מתהפך לפני שנופל	עובד ✓
E6	FlyingEnemy — שתי דריכות	דריכה ראשונה + שנייה	ראשונה מסירה כנפיים; שנייה הורגת	עובד ✓
E7	אויב נדחף לבור	אויב הגיע לקצה עם בור	נופל ונעלם	עובד ✓
E8	Koopa — צדף	דריכה על קופה	הופך לצדף	עובד ✓

📷 **צילום מסך:** גומבה מרוסק: ספרייט ריסוק ופס "STOMP" או הגדלת ניקוד על המסך

6.4 תכנית בדיקות — פריטים ואיסופים

#	מה נבדק	תנאי	תוצאה צפויה	תוצאה בפועל
C1	מטבע נאסף	נגיעה במטבע	coin counter+, נעלם	עובד ✓
C2	פטרייה מופיעה	הכאת בלוק שאלה מלמטה	פטרייה "קופצת" מהבלוק	עובד ✓
C3	פטרייה נאספת	נגיעה בפטרייה	IsSuper=true, גוף גדל	עובד ✓
C4	בלוק שאלה — מלמעלה	עמידה על בלוק שאלה	אין הפעלה (רק מלמטה)	עובד ✓
C5	בלוק שאלה — פגיעה צדדית	הליכה לתוך בלוק	נדחק הצידה	עובד ✓
C6	בלוק מנוצל — מניעת כפל	הכאת בלוק ריק	אין פריט נוסף	עובד ✓

6.5 תכנית בדיקות — Super State

#	מה נבדק	תנאי	תוצאה צפויה	תוצאה בפועל
S1	כניסה ל-Super	נגיעה בפטרייה	גוף גדל (68 → 82) פיקסל גובה	עובד ✓
S2	Super סופג פגיעה	פגיעת אויב בסופר	ירידה ל-Normal, לא נפגע	תוקן בקומיט 3cdb3fe ✓
S3			HP 1-	עובד ✓

#	מה נבדק	תנאי	תוצאה צפויה	תוצאה בפועל
	Normal פגיעה	פגיעת אויב Normal-ב		
S4	מוות ב-Super	נפילה לבור כשהוא סופר	מוות מידי	✓ עובד
S5	delta גובה נכון	BecomeNormal	גוף מצטמצם 14px בדיוק	✓ תוקן בקומיט 3cdb3fe

6.6 תכנית בדיקות — בינה מלאכותית

#	מה נבדק	תנאי	תוצאה צפויה	תוצאה בפועל
AI1	אתחול	לחיצת START	60 לואיג'ים נוצרים ונופלים על P1	✓ עובד
AI2	מוות בבור	סוכן עובר Y=560	IsAlive=false	✓ עובד
AI3	stuck detection	סוכן נתקע < 2 שני	IsAlive=false	✓ תוקן בקומיט AI
AI4	יצירת דור	כולם מתו	++Generation	✓ עובד
AI5	שיפור דורי	אחרי +10 דורות	ה-best fitness גדל	✓ נראה בניסויים
AI6	שינוי פרמטרים	שינוי NetShape ל-4,8,4,2	שגיאה אם shape לא תואמת קלט/פלט	✓ MessageBox מוצג
AI7	RESET	לחיצת RESET	Generation=0, רשתות אקראיות חדשות	✓ עובד

צילום מסך: לוח הבקרה של האימון אחרי +20 דורות: גרף best fitness עולה, או מספר הדור גבוה 

6.7 באגים שנמצאו ותוקנו

באג	תיאור	תיקון
מנהרה בנפילה	שחקן עבר דרך הרצפה בנפילה מהירה	סף נחיתה הועלה מ-25 ל-30px (1e82bb3)
קפיצה אוטומטית	החזקת Space דרך נחיתה הפעילה קפיצה שנית	jumpEdge — רק על rising edge (b67a336)
אויבים בלתי נראים	אויבים נשלחו מאחורי הרצפה	הסרת SendToBack () מפונקציות spawn (be1f398)
ריסוק בקפיצה	ריסוק אויב כשנוגעים מהצד בגובה הנכון	הוספת גרד $vy \geq 0$ לריסוק (95a0a36)
גולם כוח נפילה	נזק נפילה אחרי כל קפיצה רגילה	סף הועלה מ-60 → 120 → 220px (3cdb3fe , 5a8c95c)
גלישת int אנימציה	globalTick הגיע לערך מקסימלי אחרי שעות	globalTick % 168 (LCM של כל מחזורי אנימציה) (1e82bb3)
דליפת זיכרון HUD	TextureLoader-i fonts לא שוחררו	Dispose נוסף בכל המקומות (3cdb3fe)
JumpingEnemy קצב חצי	קפץ פעם בכל 3 שני' במקום 1.5	תיקון timer logic (d208cb7)
רשת AI שגויה	IndexOutOfRangeException כשה-shape לא תואמת 4 כניסות	ולידציה בטעינת קובץ ובשינוי שדה Shape (AI commit)

6.8 סיכום בדיקות

קטגוריה	סה"כ בדיקות	עברו	נכשלו (ותוקנו)
פיזיקה	10	10	3 (תוקנו)
אויבים	8	8	1 (תוקן)

קטגוריה	סה"כ בדיקות	עברו	נכשלו (ותוקנו)
פריטים	6	6	0
Super State	5	5	2 (תוקנו)
בינה מלאכותית	7	7	2 (תוקנו)
סה"כ	36	36	8 (תוקנו כולם)

הוראות הפעלה

7.1 דרישות מערכת

דרישה	מינימום
מערכת הפעלה	Windows 7 / 8 / 10 / 11
NET Framework.	4.7.2 ומעלה
זיכרון RAM	MB 256
כרטיס מסך	כל כרטיס עם תמיכת GDI+
מקלדת	נדרשת

NET Framework 4.7.2 מותקן כברירת מחדל ב-Windows 10 ומעלה.
למשתמשי Windows 7/8 — יש להוריד מאתר Microsoft.

7.2 התקנה והפעלה

שלב 1 — הורדת הפרויקט

- של הפרויקט ZIP-הורד את קובץ ה.
- (C:\Games\supermario: לדוגמה) חלץ לתיקייה כלשהי.

שלב 2 — בנייה (Build)

- Visual Studio ב-supermario.sln פתח את.
- Build > Build Solution (Ctrl+Shift+B) בחר.
- Output ודא שאין שגיאות בחלונית.

1. לחץ F5 (Run with debugging) או Ctrl+F5 (Run without debugging)
2. (borderless) המשחק יפתח בחלון ללא מסגרת

הערה: תיקיית `/assets/textures/sprite_sheets` חייבת להיות באותה תיקייה כמו קובץ ה-`.exe`. אם חסרה — המשחק עדיין יעבוד אך יצייר גרפיקה פרוצדורלית (+GDI) במקום ספרייט-שיטים.

 **צילום מסך:** תפריט ראשי: רקע מונפש עם כותרת, 4 כפתורים: PLAY, HOW TO PLAY, TRAIN AI, QUIT

7.3 תפריט ראשי

כפתור	פעולה
PLAY	מתחיל משחק מרמה 1
HOW TO PLAY	פותח מסך עם הסברים ומקשים
TRAIN AI	פותח ממשק אימון הבינה המלאכותית
QUIT	יוצא מהתוכנה

7.4 בקורות המשחק

מקש	פעולה
← / A	זוז שמאלה
→ / D	זוז ימינה

מקש	פעולה
Space / ↑ / W	קפץ
Escape	השהה / חזור לתפריט
Enter	המשך (כשמושהה)

טיפ לקפיצה גבוהה: החזק את מקש הקפיצה ממושך יותר — גובה הקפיצה פרופורציונלי לאורך הלחיצה.

 **צילום מסך:** מסך HOW TO PLAY עם הסבר המקשים

7.5 מכניקות המשחק

חיים וניקוד

- מתחילים עם **3 חיים**
- כל פגיעת אויב: **HP 1-**
- כשה-HP מגיע ל-0: **מוות** — חיים-- ואתחול רמה
- כשהחיים נגמרים: **Game Over** + חזרה לתפריט
- דריסת אויב: **100+ נקודות**
- איסוף מטבע: **1+ למונה המטבעות**

כוח-על (Super)

1. מצא בלוק שאלה (block ?) — ניתן לזהות לפי הצבע הצהוב
2. קפוצ **מלמטה** כדי להפעיל — תצוא פטרייה
3. גע בפטרייה — מריו יגדל (Super Mario)
4. בסופר: פגיעת אויב גורמת לכיווץ חזרה, **ללא אובדן HP**

 **צילום מסך:** מריו ב-Super state: גדול יותר, ניתן לראות את ההבדל בגובה

מעבר שלבים

- הגע לסוף הרמה (קצה ימני) כדי לעבור לשלב הבא
- לאחר שלב 5 (האחרון) — חזור לשלב 1

7.6 מצב אימון בינה מלאכותית

פתיחת מצב האימון

לחץ **TRAIN AI** מהתפריט הראשי.

 **צילום מסך:** Training Form שלם: משמאל canvas עם לואיג'ים רצים, מימין לוח בקרה

ממשק האימון

	LUIGI AI TRAINER
אזור האימון – Canvas	GENERATION: 14
לואיג'ים רצים 60	ALIVE: 23
(עם טבעת זהב על המוביל)	BEST: 1840
	ALL-TIME: 2100
	[NN Visualizer]
	SETTINGS
	Population: 60
	Mutation: 5%
	Survive: 30%

	Shape: 4, 6, 4, 2	
	[APPLY]	
	[▶ START]	
	[↺ RESET]	
	[← BACK]	

כפתורי הבקרה

כפתור	פעולה
START ▶	מתחיל את הסימולציה
PAUSE	משהה (אותו כפתור)
RESET ↺	אתחול מחדש — דור 0, רשתות אקראיות
APPLY	מחיל הגדרות חדשות ומאתחל
BACK ←	חזרה לתפריט הראשי
Escape	זהה ל-BACK

הגדרות הניתנות לשינוי

הגדרה	טווח	ברירת מחדל	השפעה
Population	10 – 500	60	גודל האוכלוסייה בכל דור
Mutation %	1 – 50	5	הסתברות לשינוי משקולה
% Survive	5 – 80	30	אחוז הסוכנים ששורדים לדור הבא
Net Shape	מחרוזת	4, 6, 4, 2	מבנה שכבות הרשת — חייב להתחיל ב-4 ולסיים ב-2

אזהרה: שינוי ה-Net Shape לערך לא תואם (לדוגמה: 3, 6, 2) יגרום להודעת שגיאה. הספרה הראשונה **חייבת להיות 4** (מספר הקלטים) והאחרונה **חייבת להיות לפחות 2** (מספר הפלטים).

קריאת היוזאליזציה

- **נקודות** = נויורונים. צהוב בהיר = אקטיבציה גבוהה
- **קווים כחולים** = משקולה חיובית (מחזקת)
- **קווים אדומים** = משקולה שלילית (מעכבת)
- **עובי הקו** = גודל המשקולה המוחלט

 **צילום מסך:** יוזאליזציה של רשת הנוירונים בזמן ריצה: 4 נויורוני קלט, 2 שכבות נסתרות, 2 פלטים

כיצד לדעת שהסוכן לומד?

- ה-BEST וה-ALL-TIME BEST צריכים לעלות לאורך הדורות
- בדורות הראשונים — רוב הלואיג'ים נופלים לבור מיד
- אחרי 15-30 דורות — חלק מהם מצליחים לחצות מספר פלטפורמות
- מוביל בולט = לואיג'י עם **טבעת זהב** שהמצלמה עוקבת אחריו

סיכום ומסקנות

8.1 מה הושג

משחק

בניתי משחק פלטפורמה פונקציונלי ואמין בסגנון Super Mario, הכולל:

- מנוע פיזיקה עצמאי עם 6 קבועים כוונונים, קפיצה משתנה, פתרון התנגשויות AABB
- 6 סוגי אויבים עם התנהגות שונה: הלכה, קפיצה, סיור, טיסה
- 3 שלבים עיצוביים + 2 פרוצדורליים מ-25 תבניות
- מנגנוני כוח-על (פטרייה, מצב Super), מטבעות, בלוקי שאלה
- תפריט ראשי מונפש, HUD, מסך pause, מוות מונפש
- ספרייט-שיטים אותנטיים + fallback GDI + פרוצדורלי

בינה מלאכותית

פיתחתי מנוע Neuroevolution שלם:

- רשת נוירונים שכבתית: `Neuron → Layer → NeuralNetwork`
- אלגוריתם גנטי עם elitism, crossover ומוטציה
- סוכן MarioAgent עם פיזיקה זהה לשחקן ו-4 חיישנים
- ויזואליזציה חיה של הרשת בכל tick
- ממשק אימון מלא עם הגדרות כוונון

8.2 קשיים שנתקלתי בהם

קשי 1 — "מנהרת" פיזיקה (Tunneling)

הבעיה: שחקן שנפל במהירות גבוהה ($15\text{px/tick} <$) "עבר" דרך הרצפה.

הפתרון: העלאת סף הנחיתה מ-25 ל-30px. ה"קסם" — הבדיקה אינה "האם גוף בתוך פלטפורמה?" אלא "האם חדירת הגג קטנה מ-30px?". כל עוד הגוף זז לפחות כפריים אחד (16ms) לא יחמיק.

לקח: ב-discrete physics (לא continuous) יש להתחשב בגודל הצעד המקסימלי.

קשי 2 — ביצועים של WinForms

הבעיה: המשחק ירד ל-30fps כשהיו 75 PictureBox-ים לרצועת הקרקע.

הפתרון: החלפת 75 PictureBox נפרדים בפקד אחד ברוחב 3000px. בנוסף, `SuspendLayout / ResumeLayout()` עטפו את כל `PhysicsStep` ב-pair אחד במקום 7.

לקח: ב-WinForms, `Layout Reflow` הוא יקר — כל `PictureBox` מפעיל אותו. מינימיזציה של `controls`.

קשי 3 — שחקן מסיים את הקפיצה בכניסה לשלב

הבעיה: אחרי מוות ואתחול, השחקן פעמים רבות קפץ אוטומטית מיד.

הסיבה: מקש הקפיצה היה מוחזק בעת המוות, ובאתחול `jumpEdge` לא אופס.

הפתרון: מעבר מ-"Is the key down?" ל-"Was there a rising edge?". כך החזקת מקש לא מספיקה — צריך לשחרר ולחזור ללחץ.

קשי 4 — Neuroevolution לא מתכנסת

הבעיה: הסוכנים לא שיפרו את עצמם גם אחרי 100 דורות.

הסיבה א': `stuck detection` לא עבד — סוכן שנתקע נשאר חי לנצח וה"fitness" שלו היה תמיד 0.

הסיבה ב': הקלטים (inputs) היו מחושבים בטעות בקואורדינטות מסך (screen) ולא בקואורדינטות עולם (world), כך שה-camera offset שינה את הקלטים באופן לא עקבי.

הפתרון: תיקון שני הבאגים. אחרי התיקון — שיפור ניכר כבר בדור 5.

8.3 מה הייתי עושה אחרת

1. **Unit Tests:** לכתוב בדיקות אוטומטיות לפיזיקה. עם WinForms קשה, אך ניתן לבודד את `Player.cs` כ-class שלא תלוי ב-Form ולבדוק אותו ב-MSTest.

2. **Save/Load לרשת:** לשמור את הרשת הטובה ביותר לקובץ JSON/binary כדי

שאפשר יהיה לטעון אותה בהפעלה הבאה מבלי לאמן מחדש.

3. **מנוע Entity-Component:** לפתח ארכיטקטורה (Entity-Component-System) ECS

כדי לצמצם כפילויות קוד בין 6 סוגי האויבים.

4. **מוסיקה:** להוסיף ניגון bit-8 בסגנון NES.

8.4 הרחבות אפשריות

הרחבה	תיאור
שמירת התקדמות	שמירת שלב + ניקוד לקובץ
Multi-player	שחקן 2 — לואיג'י בשליטת שחקן שני
עורך שלבים	ממשק גרפי לבניית שלבים
עוד סוגי אויבים	Piranha Plant בתוך צינורות, Bullet Bill
AI מתקדמת	Reinforcement Learning (Q-Learning / PPO) במקום Neuroevolution
אימון בסביבה מלאה	הכנסת אויבים ומטבעות לזירת האימון
Leaderboard	שמירת שיאים מקומיים

8.5 מסקנות

הפרויקט לימד אותי שלוש תובנות מרכזיות:

1. **פשטות < מורכבות:** לא השתמשתי במנוע משחקים, ב-Entity Framework או ב-ML library. כתיבת הכל מאפס איפשרה לי להבין לעומק כל מרכיב — מהפיזיקה ועד הגנטיקה.

2. **באגים הם הזדמנויות:** כל אחד מהבאגים שתיקנתי (tunneling, auto-jump, invisible enemies, AI inputs) לימד אותי עיקרון חדש שלא הייתי מבין ממקרא ספר.

3. **Neuroevolution עובד גם ללא ידע מוקדם:** הסוכן לא "יודע" שצריך לקפץ מעל בורות. הוא רק יודע שצריך ללכת ימינה. מהעיקרון הפשוט הזה, בשילוב אלגוריתם גנטי, מתפתח התנהגות חכמה.

Kent Beck — *"Make it work, then make it right, then make it fast"*

ביבליוגרפיה ומקורות

ספרים ומאמרים

1. **Millington, I., & Funge, J.** (2009). *Artificial Intelligence for Games* (2nd ed). Morgan Kaufmann.
(ניוראבולוציה, אלגוריתמים גנטיים בסביבות משחק)
2. **Nielsen, M. A.** (2015). *Neural Networks and Deep Learning*. Determination Press.
גיש באינטרנט: <http://neuralnetworksanddeeplearning.com>
3. **Floreano, D., & Mattiussi, C.** (2008). *Bio-Inspired Artificial Intelligence*. MIT Press.
(אבולוציה מלאכותית ו-Neuroevolution)

תיעוד טכני

1. **Microsoft Docs** — *Windows Forms in .NET Framework 4.7.2*
<https://learn.microsoft.com/en-us/dotnet/framework/winforms>
2. **Microsoft Docs** — *Graphics Class (System.Drawing)*
<https://learn.microsoft.com/en-us/dotnet/api/system.drawing.graphics>
3. **Microsoft Docs** — *Partial Classes and Methods (C#)*
<https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/partial-classes-and-methods>

מקורות על פיזיקת משחקי פלטפורמה

1. **Celeste & TowerFall developer, Noel Berry** — *Game Feel and Responsive Controls*
(על חשיבות תאוצות מול velocity-set)

2. **The Mechanics of Super Mario Bros** — ניתוח פיזיקה קלאסי (StackExchange)
(GameDev / Mario Wiki)

3. **AABB Collision Detection** — GameDev.net
(*Separating Axis Theorem* וגישת *ResolveSmallestOverlap*)

כלי פיתוח

1. **Python Pillow (PIL)** — ספריית עיבוד תמונה לייצור ה-sprite sheets
<https://pillow.readthedocs.io>

2. **Git & GitHub** — ניהול גרסאות
<https://github.com/amitamit10/supermario>

השראה וייחוס

1. **Nintendo** — *Super Mario Bros* (1985) — על עיצוב המכניקה, תחושת הפיזיקה והספרייט המקורי שממנו הופק ה-pixel art.

כל השימוש בחומרים מוגני זכויות יוצרים הוא למטרות לימודיות בלבד, במסגרת פרויקט בגרות.